

INTRO TO LINEAR AGENTS + CLAUDE

June 2026

Searched available tools >

Let me pull up recent activity for you.

List Issues >

Here's the current state of your **Groot279** workspace, mostly centered on the **Mobile App Redesign** project:

Most recently active:

- **GRO-12** - "Face ID broken - not sure why" — just updated today, no priority set, in Backlog
- **GRO-6** - "App freezes on launch after splash screen" — **Urgent**, Backlog
- **GRO-11** - "Login crashes when using Face ID" — no priority, Backlog

Other open issues in Mobile App Redesign:

- **GRO-5** - App crashes on login with biometric auth (**Urgent**, Todo)
- **GRO-7** - Add biometric login for returning users (High, Backlog)
- **GRO-10** - Add dark mode support (High, Backlog)
- **GRO-8** - Improve dashboard load time on mobile (Medium, Backlog)
- **GRO-9** - Profile image upload fails on poor network (Low, Backlog)

Worth flagging: there are now **three separate issues** (GRO-12, GRO-11, GRO-5) all pointing at biometric/Face ID login problems — they look like they might be duplicates or closely related. Want me to take a closer look at those and possibly link or merge them?

Also still sitting untouched: the four onboarding issues (GRO-1 through GRO-4 — "Get familiar with Linear," "Connect your tools," etc.), all in Todo since the workspace was created.

Project Overview

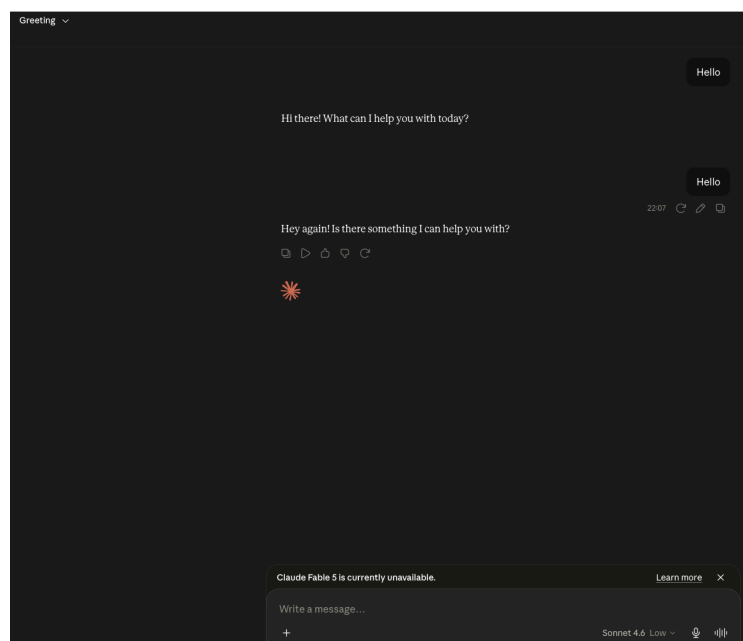
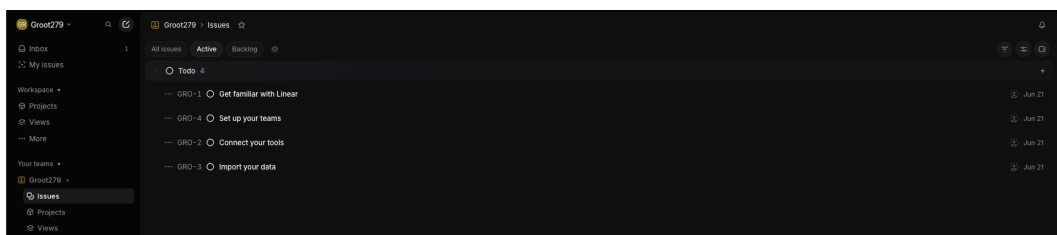
What this project set out to achieve

I'm setting up a complete AI-driven project management system using Linear, Linear Agent, and Claude Desktop connected through the Model Context Protocol. My goal is to turn a messy backlog of bugs, features, and performance complaints into a streamlined workflow I can control through plain conversation. I'll create a realistic Linear workspace with sample issues, configure Linear Agent to handle create, update, and summarize tasks, enable Triage Intelligence during a free Business trial to catch duplicates, and connect Claude Desktop so I can manage everything from outside Linear. As a bonus, I'll build a custom Skill for automating weekly status reports, saving me hours of manual sorting.

Setting Up Linear and Claude Desktop

Step goals

In this step, I'm setting up my foundational tools, Linear and Claude Desktop, so that I can build the AI-powered project management workflow for the rest of the project. I'll create a fresh Linear workspace to host my project board and sample issues, install Claude Desktop as my AI conversation interface, and verify both are properly installed and ready to connect. Getting these basics right ensures that when I later add Linear Agent and MCP connections, everything works smoothly and I can truly manage my backlog through natural language instead of manual clicks.



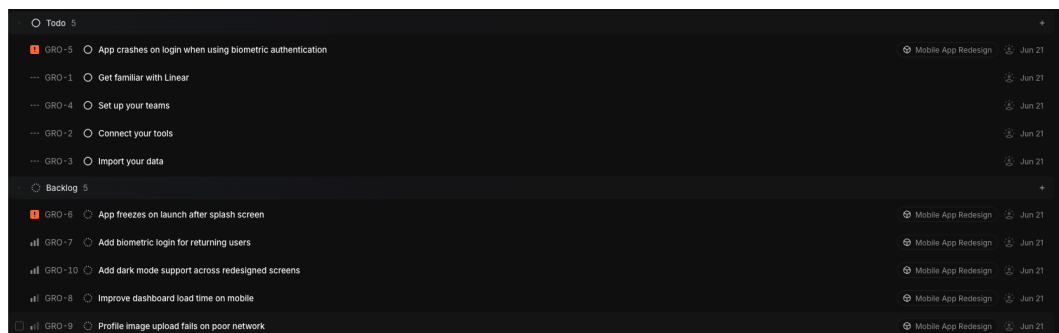
Tools and their roles in the project

I set up Linear, which will handle storing and organizing all my project issues, backlogs, and boards as the central source of truth. I also set up Claude Desktop, which will let me manage that entire project board through natural language conversations, creating, updating, and summarizing tickets without ever clicking through the Linear interface directly.

Building a Structured Project with Issues and Priorities

Step goals

In this step, I'm setting up a realistic project backlog inside Linear so that I can give the AI features meaningful data to manage. I'll create a dedicated project for my mobile app, then use Linear Agent to batch-create a set of issues including bugs, feature requests, and performance complaints. I'll organize everything with labels for priority and category so the board is structured and ready. Once this backlog is populated, I can test the AI's ability to sort, summarize, and update work through natural conversation, turning a chaotic list into an actionable workflow.



Why priority levels matter

I created issues at different priority levels because real projects always have a mix of urgent bugs, important features, and nice-to-have improvements. This variety lets me practice triage and see how AI handles sorting and prioritizing work naturally. If everything were the same priority, I wouldn't get a realistic sense of how Linear Agent can help me manage a chaotic backlog, and I wouldn't be able to test smart routing or duplicate detection effectively later on.

Worked for 15 seconds

All 6 issues in  Mobile App Redesign already have one of those labels, and each one looks like the best fit, so I didn't make any changes.

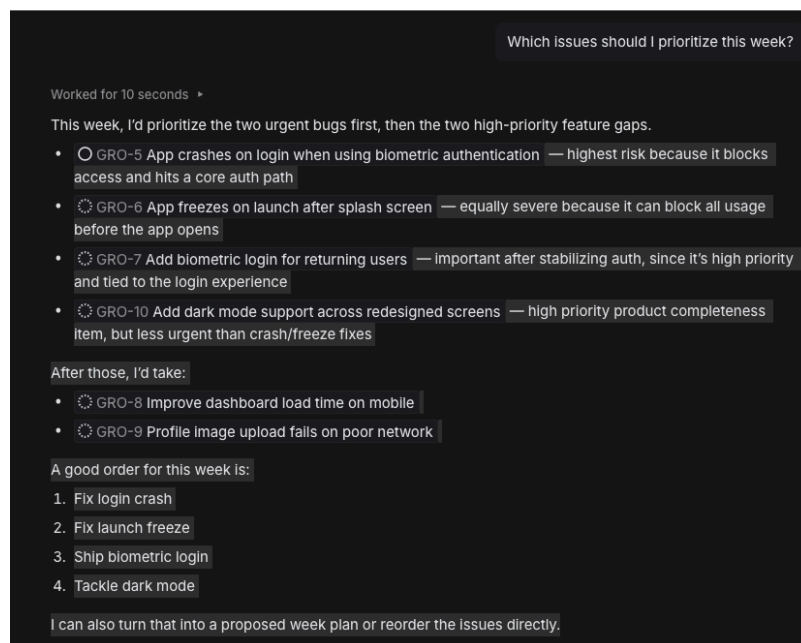
	GRO-5		App crashes on login when using biometric authentication	
	GRO-6		App freezes on launch after splash screen	
	GRO-7		Add biometric login for returning users	
	GRO-8		Improve dashboard load time on mobile	
	GRO-9		Profile image upload fails on poor network	
	GRO-10		Add dark mode support across redesigned screens	

If you meant a different project by "your project or app name project," tell me which project and I'll apply the labels there.

Using Linear Agent for AI-Powered Project Management

Step goals

In this step, I'm putting Linear Agent to work on my real project backlog so that I can experience AI-powered project management through natural conversation. I'll open the agent chat and query my existing issues to test its understanding of my workspace. Then I'll ask for a strategic summary of my project's status and a risk analysis to identify potential blockers. Finally, I'll request AI-generated prioritization advice to help me decide what to tackle first. This step proves that I can move from manual sorting to conversational management, letting the AI handle the heavy lifting of analysis so I can focus on decision-making.



What Linear Agent can do

I can use Linear Agent to create, update, summarize, and prioritize issues through natural conversation. For example, I asked it to batch-create five issues with different priorities and labels, and it handled that in seconds without me opening multiple forms. I also asked for a project summary and got a clear risk breakdown. I can see this being useful for daily standups, where I can quickly ask what's blocking progress or what urgent bugs need attention, without clicking through dashboards. It saves me time on repetitive tasks and helps me stay focused on decisions rather than data entry.

Evaluating the AI's project assessment

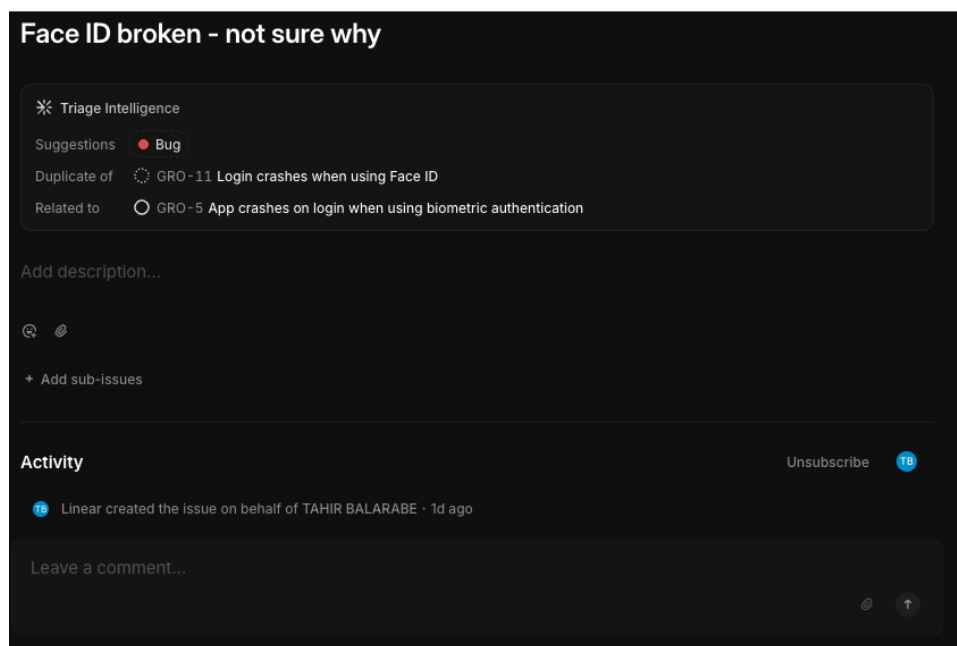
I agree with the summary because it accurately highlights that my project is marked "In Progress" but has zero active work, with all issues still in Backlog or Todo. The risk assessment around delivery, quality, and planning feels spot on for a startup scenario.

I would change the assessment by adding more specific action items, like recommending which single issue to start first based on impact versus effort. I'd also ask it to flag dependencies, like whether the biometric login bug blocks the onboarding redesign. Adding a timeline estimate for when to hit first milestone would make the summary more actionable for my team.

Exploring Triage Intelligence and Duplicate Detection

Step goals

In this step, I'm enabling Linear's Triage Intelligence feature so that I can experience AI-powered pattern detection across my project backlog. I'll verify it's active in my workspace settings, then intentionally create duplicate issues to trigger the system's duplicate detection capability. Once Triage Intelligence surfaces its suggestions, I'll review and act on them, seeing firsthand how the AI identifies overlapping work and recommends smart routing. This step moves me from manual issue management to proactive AI assistance that catches problems I might otherwise miss, saving time and reducing redundancy in my workflow.



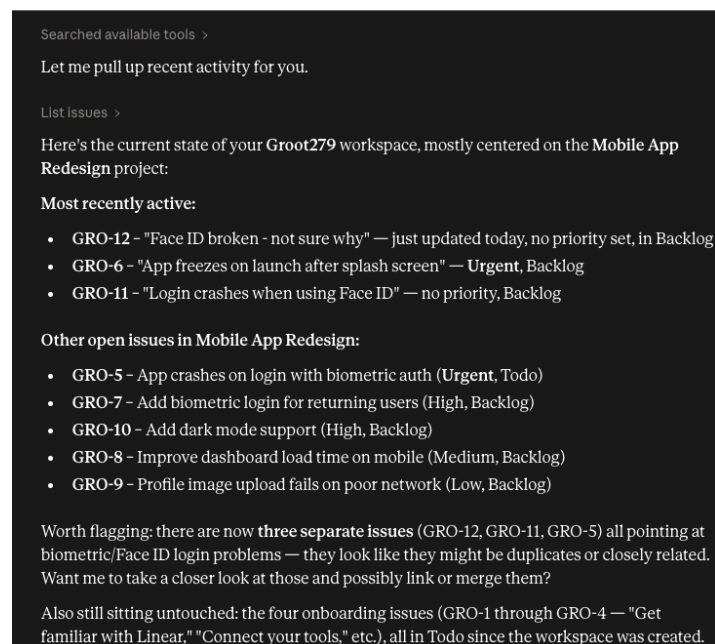
What Triage Intelligence suggested

When I ran Find Suggestions, Triage Intelligence showed a duplicate suggestion linking my new "Face ID broken" issue to my existing "Login crashes when using Face ID" issue, plus a related link to "App crashes on login using biometric authentication." I chose to apply the duplicate link to the most similar issue, which automatically created a relation in the right sidebar. I also saw the activity recorded in the issue timeline. This confirmed that the AI semantically matched the wording even though I phrased it differently, proving it catches redundant reports I might have missed manually.

Connecting Linear to Claude via MCP

Step goals

When I ran Find Suggestions, Triage Intelligence showed a duplicate suggestion linking my new "Face ID broken" issue to my existing "Login crashes when using Face ID" issue, plus a related link to "App crashes on login using biometric authentication." I chose to apply the duplicate link to the most similar issue, which automatically created a relation in the right sidebar. I also saw the activity recorded in the issue timeline. This confirmed that the AI semantically matched the wording even though I phrased it differently, proving it catches redundant reports I might have missed manually.



Searched available tools >

Let me pull up recent activity for you.

List issues >

Here's the current state of your **Groot279** workspace, mostly centered on the **Mobile App Redesign** project:

Most recently active:

- GRO-12 - "Face ID broken - not sure why" — just updated today, no priority set, in Backlog
- GRO-6 - "App freezes on launch after splash screen" — **Urgent**, Backlog
- GRO-11 - "Login crashes when using Face ID" — no priority, Backlog

Other open issues in Mobile App Redesign:

- GRO-5 - App crashes on login with biometric auth (**Urgent**, Todo)
- GRO-7 - Add biometric login for returning users (High, Backlog)
- GRO-10 - Add dark mode support (High, Backlog)
- GRO-8 - Improve dashboard load time on mobile (Medium, Backlog)
- GRO-9 - Profile image upload fails on poor network (Low, Backlog)

Worth flagging: there are now **three separate issues** (GRO-12, GRO-11, GRO-5) all pointing at biometric/Face ID login problems — they look like they might be duplicates or closely related. Want me to take a closer look at those and possibly link or merge them?

Also still sitting untouched: the four onboarding issues (GRO-1 through GRO-4 — "Get familiar with Linear," "Connect your tools," etc.), all in Todo since the workspace was created.

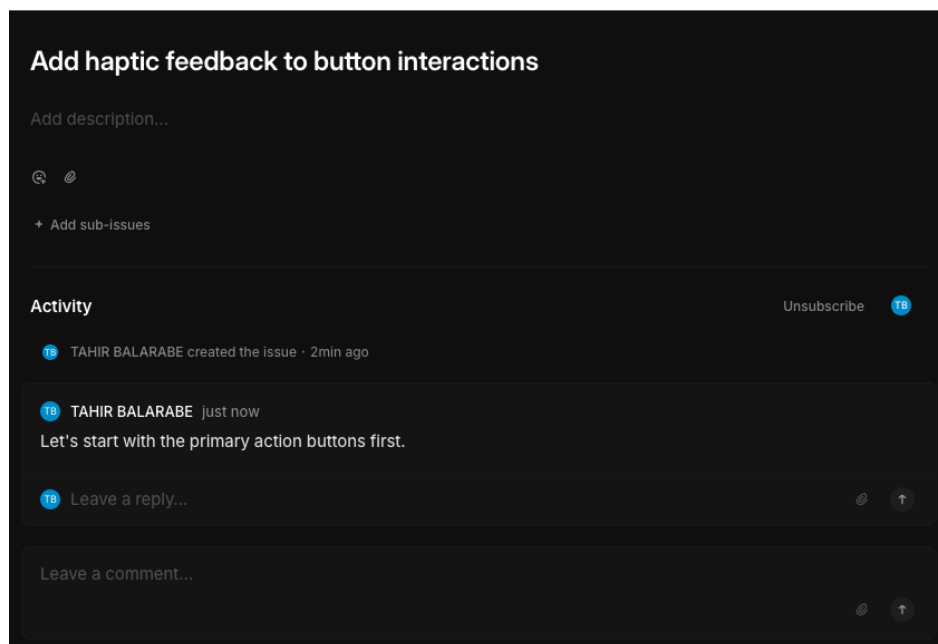
Understanding MCP and how it works

MCP, or Model Context Protocol, is the underlying technology that lets Claude communicate directly with external applications like Linear. It acts as a shared language and secure bridge between Claude and my workspace. It lets Claude access my Linear workspace because I authorized a connection through OAuth, which gives Claude a limited access token to read and update my issues on my behalf. This means Claude can query my project, list issues, create new ones, and add comments, all through natural conversation, without ever storing my password or seeing data I haven't permitted.

Managing Linear Directly from Claude Desktop

Step goals

In this step, I'm using Claude Desktop to actively manage my Linear workspace through natural conversation, so that I can experience the full power of MCP integration. I'll start by querying my project with different prompts to test Claude's ability to pull specific data, like listing all urgent bugs or showing issues without assignees. Then I'll create a brand new issue and add a comment to it, all from within Claude Desktop, without ever opening Linear. This step proves that I can truly offload project management tasks to my AI assistant, keeping me in my flow state while Claude handles the administrative work behind the scenes.



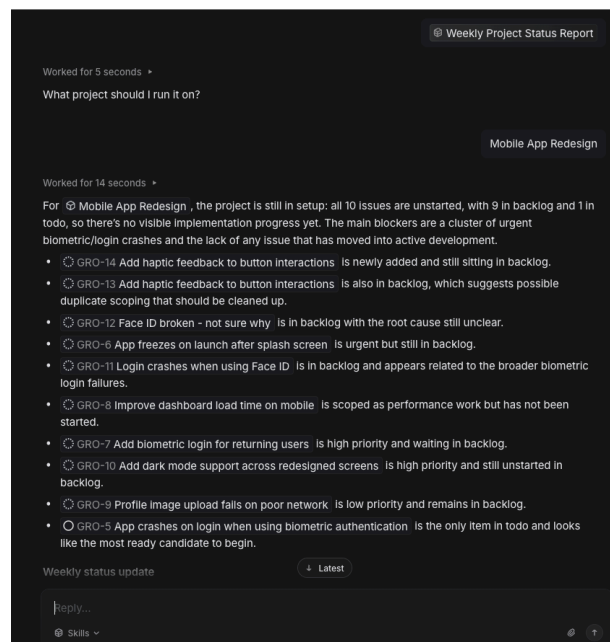
Creating issues and comments through Claude

When I asked Claude to create an issue, it successfully generated a new ticket titled "Add haptic feedback to button interactions" with medium priority and placed it in my Mobile App Redesign project, then gave me a clickable link to verify it. When I added a comment, it found that same issue and appended my note about starting with primary action buttons, confirming the update in the chat. Both actions happened instantly without me opening Linear, and I could see the new issue and comment reflected in my workspace when I refreshed. This proved that Claude can both read and write to my project board through natural language commands.

Automating Workflows with Custom Skills

Project extension goals

In this project extension, I'm creating a custom Linear Agent Skill for automated weekly status reports, so that I can save time and eliminate manual effort every Monday morning. Instead of opening Linear, clicking through issues, and mentally tallying progress, I'll write a structured prompt that pulls live project data, identifies blockers, and formats everything into a ready-to-send update. I'll save this prompt as a reusable Skill in Linear Agent, then test it with a single slash command to confirm it produces consistent, actionable results. This turns a repetitive ten-minute task into a one-click automation, letting me focus on decisions instead of data gathering.



The Skill created and why

In this project extension, I saved a prompt that generates a weekly status report for my Mobile App Redesign project, because this is the most repetitive and time-consuming task I face every Monday morning. The prompt asks Linear Agent to list all issues, summarize each in one sentence, identify blockers, and format everything into clear sections for Progress, Blockers, and Next Steps. I chose this specific workflow to automate because it saves me ten minutes of manual clicking and tallying each week, ensures consistent formatting for my team lead, and reduces the mental overhead of remembering what moved forward. Now I can run it with a single slash command and focus on decision-making instead of data gathering.

Key Takeaways and Reflections

Tools and concepts learned

The key tools I used include Linear for project management, Linear Agent for in-app AI assistance, Claude Desktop for external AI interaction, and MCP for secure integration between them. I also used Triage Intelligence for duplicate detection and Skills for automating recurring workflows.

Key concepts I learnt include using natural language to batch-create and update issues, querying project data conversationally, applying labels for better categorization, and saving reusable prompts as slash commands. I also understood how MCP acts as a bridge, letting Claude read and write to Linear without storing credentials. Most importantly, I learnt that AI can handle repetitive administrative work so I can focus on strategic decisions.

Time and challenges

This project took me approximately 50 to 55 minutes to complete, slightly over the estimated 45 minutes. The most challenging part was getting Triage Intelligence to detect duplicates consistently, as the initial indexing took a few minutes and required me to refresh and rerun the Find Suggestions command. I also found fine-tuning my Skill prompt tricky, balancing enough detail to get consistent formatting without making it too rigid. Overall, the MCP connection setup was smoother than expected, and the time investment paid off with a reusable automation that will save me time every week.

Final reflections

I did this project today to learn how to combine Linear, Linear Agent, and Claude Desktop with MCP to manage a software backlog through natural language, instead of clicking through menus manually. Another skill I want to learn is building more advanced custom Skills that can trigger multi-step workflows, like automatically moving issues to In Progress when blockers are resolved or generating sprint planning documents from my backlog. I also want to explore integrating other tools like Slack or GitHub into the same MCP ecosystem to create a fully connected AI-powered development workflow. This project gave me a solid foundation, and I'm excited to expand it further.